Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 5

Part A (To be referred by students)

NumPy

SAVE THE FILE AND UPLOAD AS (RollNo_Name_Exp5)

Aim: To understand and apply basic and advanced NumPy operations and built-in matrixfunctions, focusing on their application in data science and machine learning tasks.

Problem Statement:

Solve the following computational problems by **recognizing** python data structures of NumPy and **apply** them to implement programs

1. Create a 3x3 matrix of random integers between 1 and 10 and find the total sum of its elements.
2. Generate a 4x4 matrix of random numbers and compute the average of all its elements.
3. Create a 2x3 matrix and find the sum of all elements in the second row.
4. Generate a 5x5 matrix of random integers and determine the highest value in each column.
5. Create a 4x4 matrix and calculate the sum of its main diagonal elements.
6. Generate a 3x3 matrix and find the smallest value among all its elements.
7. Create a 4x4 matrix of random integers between 1 and 20 and compute the sum of all elements that are greater than 10.
8. Generate a 5x5 matrix of random numbers and find the mean of the maximum values of each row.
9. Create a 3x3 matrix and calculate the difference between the sum of the elements in the first row and the sum of the elements in the last row.
10. Generate a 6x6 matrix of random integers and replace all values below the median with zero.
11. Create a 4x4 matrix and compute the sum of elements in the upper triangular part of the matrix.
12. Generate a 3x4 matrix of random values and find the standard deviation of the elements in the third column.

Topic covered: Numpy

Learning Objective: Learner would be able to

1. Learn how to import packages in python
2. Features and available functions of Numpy

3. Create multi-dimensional data structures and manipulate them

Theory:

- NumPy is 'Numerical Python package consisting of multidimensional array (ndarray) objects and a collection of routines for processing of array.

#Numpy with one dimension import numpy as np a = np.array([1,2,3]) print(a) O/P - [1, 2, 3]	# more than one dimensions import numpy as np a = np.array([[1, 2], [3, 4]]) print(a) O/P - [[1, 2] [3, 4]]

NumPy's array class is called ndarray. The important attributes of an ndarray object are:
ndarray.ndim - the number of axes (dimensions) of the array.
ndarray.shape - the dimensions of the array. This is a tuple of integers indicating the size of the array in each dimension. For a matrix with n rows and m columns, shape will be (n,m). The length of the shape tuple is therefore the number of axes, ndim.
ndarray.size -the total number of elements of the array. This is equal to the product of the elements of shape.
ndarray.dtype -an object describing the type of the elements in the array. One can create or specify dtype's using standard Python types. Additionally, NumPy provides types of its own. numpy.int32, numpy.int16, and numpy.float64 are some examples.
ndarray.itemsize - the size in bytes of each element of the array. For example, an array of elements of type float64 has itemsize 8 (=64/8), while one of type complex32 has itemsize 4 (=32/8). It is equivalent to ndarray.dtype. itemsize.
ndarray.data - the buffer containing the actual elements of the array. Normally, we won't need to use this attribute because we will access the elements in an array using indexing facilities.
Example –

import numpy as np a = np.arange(15).reshape(3, 5) print(a) array([[0, 1, 2, 3, 4], [5, 6, 7, 8, 9], [10, 11, 12, 13, 14]]) print(a.shape) (3, 5) print(a.ndim) 2 print(a.dtype.name) 'int64'	print(a.itemsize) 8 print(a.size) 15 print(type(a)) <class 'numpy.ndarray'> print(b = np.array([6, 7, 8]) print(b) array([6, 7, 8]) print(type(b)) <class 'numpy.ndarray'>

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

Array Creation There are several ways to create arrays. For example, you can create an array from a regular Python list or tuple using the array function

import numpy as np a = np.array([2, 3, 4]) print(a) array([2, 3, 4]) print(a.dtype) dtype('int64') print(b = np.array([1.2, 3.5, 5.1]) print(b.dtype) dtype('float64')	print(a = np.array(1, 2, 3, 4) # WRONG Traceback (most recent call last): ... TypeError: array() takes from 1 to 2 positional arguments but 4 were given print(a = np.array([1, 2, 3, 4]) # RIGHT	b = np.array([(1.5, 2, 3), (4, 5, 6)]) print(b) array([[1.5, 2. , 3.], [4, 5 , 6]]) **The type of the array can also be explicitly specified at creation time:** print(c = np.array([[1, 2], [3, 4]], dtype=**complex**) print(c) array([[1.+0.j, 2.+0.j], [3.+0.j, 4.+0.j]])

To create sequences of numbers, NumPy provides the arange function which is analogous to the Python built-in range, but returns an array.
print(np.arange(10, 30, 5))
array([10, 15, 20, 25])
print(np.arange(0, 2, 0.3)) # it accepts float arguments
array([0. , 0.3, 0.6, 0.9, 1.2, 1.5, 1.8])

Universal Functions
NumPy provides familiar mathematical functions such as sin, cos, and exp

B = np.arange(3) print(B) array([0, 1, 2]) print(np.exp(B)) array([1. , 2.71828183, 7.3890561])	print(np.sqrt(B)) array([0. , 1. , 1.41421356]) print(C = np.array([2., -1., 4.]) print(np.add(B, C)) array([2., 0., 6.])

Changing the **shape** of an array

a = np.floor(10 * rg.random((3, 4)))
print(a)
array([[3., 7., 3., 4.],
[1., 4., 2., 2.],
[7., 2., 4., 9.]])
print(a.shape)
(3, 4)

The shape of an array can be changed with various commands. Note that the following three commands all return a modified array, but do not change the original array:

```
print( a.ravel()) # returns the array, flattened
array([3., 7., 3., 4., 1., 4., 2., 2., 7., 2., 4., 9.])
print( a.reshape(6, 2)) # returns the array with a modified shape
array([[3., 7.],
[3., 4.],
[1., 4.],
[2., 2.],
[7., 2.],
[4., 9.]])
print( a.T) # returns the array, transposed
array([[3., 1., 7.],
[7., 4., 2.],
[3., 2., 4.],
[4., 2., 9.]])
print( a.T.shape)
(4, 3)
print( a.shape)
(3, 4)
```

Department of AI_DS
Academic Year: 2026-27 (Odd Sem)
Course: Python for Data Science Lab
Second Year B. Tech Semester: III

PRACTICAL 5

Part B (to be completed by students)

Numpy

1. All the students are required to perform the given tasks in Jupyter Notebook
2. Create a new notebook for each experiment. The filename should be RollNo_Name_Exp5)
3. In the first cell, the student must write his/her Name, roll no and class in the form of comments
4. Every program should be written in separate cells and in the given sequence
5. After completing the experiment, download the notebook in pdf format. The filename should be RollNo_Name_Exp5.pdf).
   Note: - To download as a PDF, go to File and choose Print Preview. This will open a new tab. Now, press Ctrl + P and opt to save as PDF.
6. Upload the pdf on the web portal